

<JS>

JavaScript — основы и особенности языка

значения, объекты, функции, замыкания
и асинхронность

Дробышев Максим

старший разработчик · Яндекс Еда

app.js

```
1 const orders = [  
2   { amount: 900, paid: true },  
3   { amount: 650, paid: false },  
4 ];  
5  
6 const total = orders  
7   .filter(order => order.paid)  
8   .reduce((sum, order) => sum + order.amount, 0);
```

[]

{ }

=>

P

CALL STACK

main ()

MICROTASKS

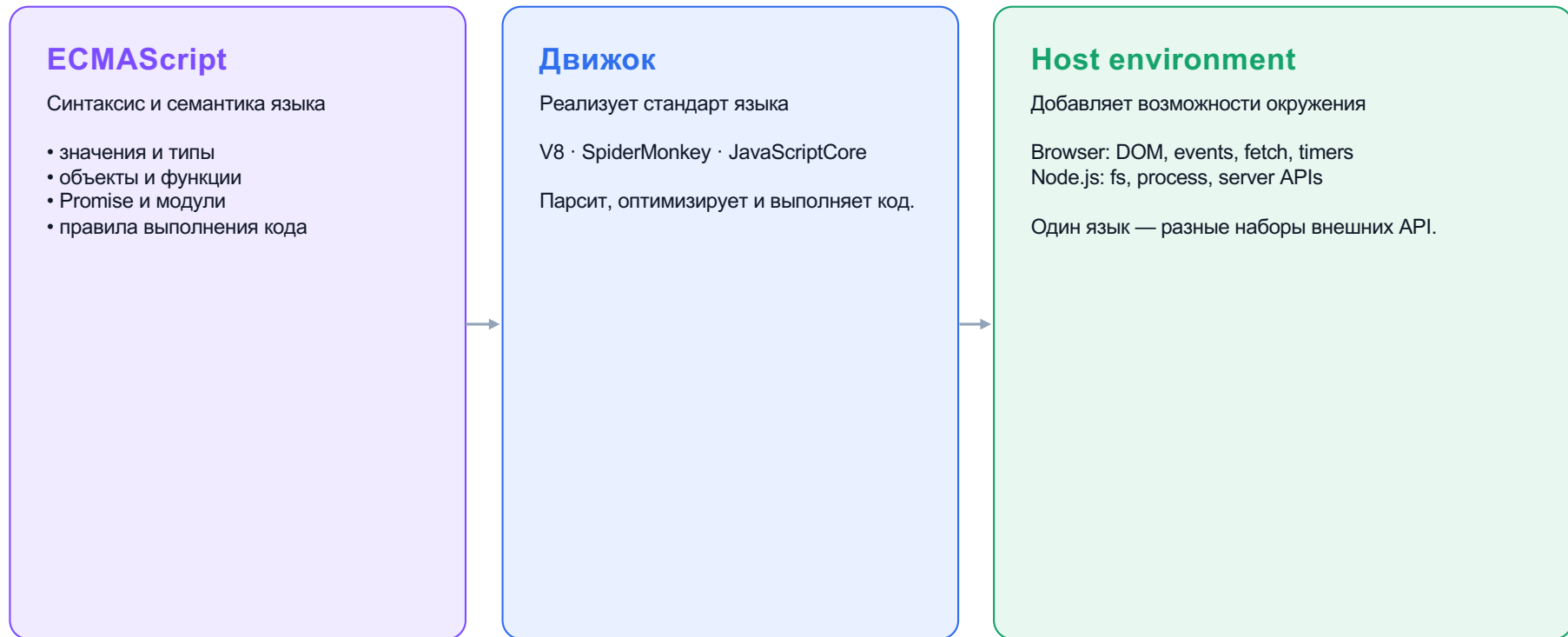
then ()

Что успеем за 90 минут

01	Среда выполнения и подключение к странице	RUNTIME	0–15 мин
02	Значения, типы и преобразования	VALUES	15–35 мин
03	Объекты, массивы и прототипы	DATA	35–55 мин
04	Функции, области видимости, замыкания и this	FUNCTIONS	55–75 мин
05	Ошибки, event loop и async/await	ASYNC	75–90 мин

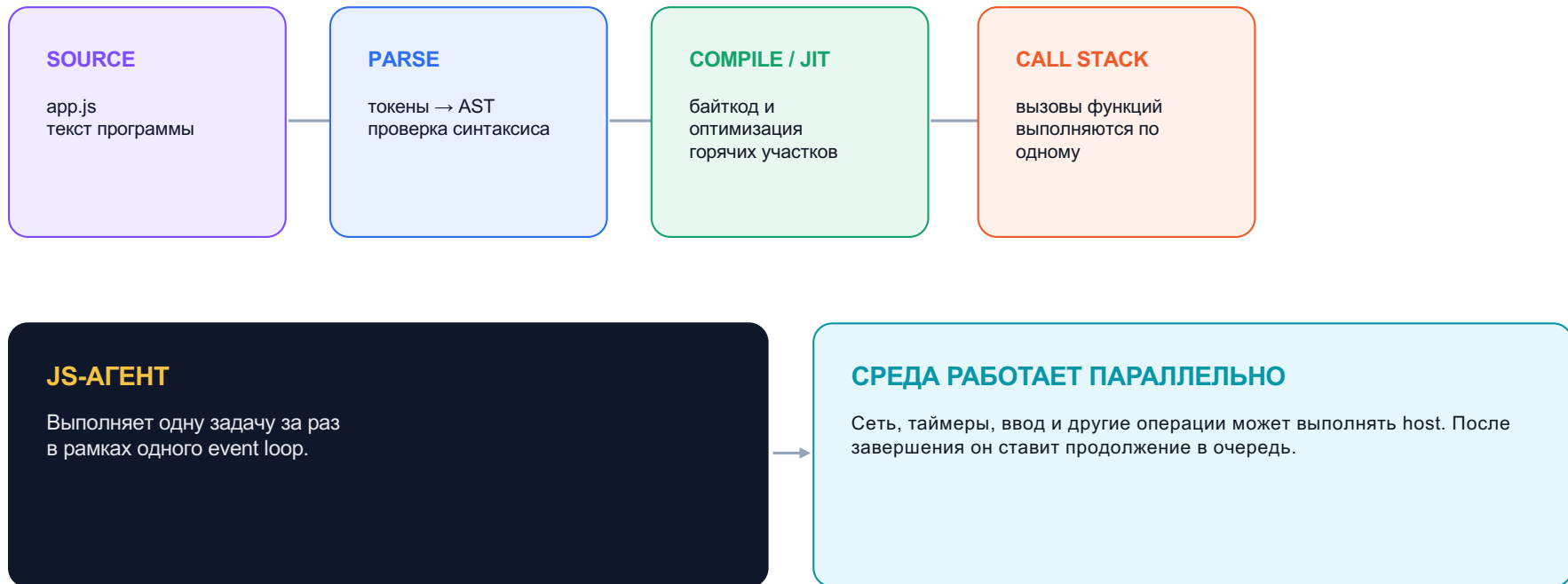
По ходу — короткие вопросы аудитории, DevTools и мини-практика «предскажите вывод».

JavaScript = язык + среда выполнения



Важно: document и fetch — не «синтаксис JavaScript», а API среды.

От исходного кода до выполнения



«Однопоточный JS» не означает, что весь браузер однопоточен.

classic script, defer, async и modules

index.html

```
1 <head>
2   <script src="legacy.js" defer></script>
3   <script type="module" src="app.js"></script>
4 </head>
```

defer

Загрузка параллельно.
Выполнение после разбора HTML.
Порядок сохраняется.

async

Выполнение сразу после загрузки.
Порядок между файлами не гарантирован.

module

defer по умолчанию, strict mode, собственная область видимости, import/export.

app.js

```
1 // app.js — module scope + strict mode
2 import { loadOrders } from './orders.js';
3
4 const orders = await loadOrders();
5 console.log(orders);
```

ПРАКТИЧЕСКОЕ ПРАВИЛО

Для нового учебного проекта используйте type="module".

Classic + defer нужен, если работаете с обычным скриптом без import/export.

async — для независимых скриптов аналитики и виджетов.

const и let — по умолчанию; var — историческое наследие

const

Блочная область видимости
Нельзя переназначить binding
Есть temporal dead zone

Используйте по умолчанию.

let

Блочная область видимости
Можно переназначить
Есть temporal dead zone

Когда значение действительно меняется.

var

Функциональная, а не блочная scope
Поднимается как undefined
Может создавать свойства global object

Избегайте в новом коде.

```
scope.js
1 for (let i = 0; i < 3; i += 1) {
2   const square = i ** 2;
3   console.log(square);
4 }
5
6 // console.log(i); // ReferenceError
```

const фиксирует привязку, но не делает объект неизменяемым

```
const.js
1 const settings = {
2   theme: 'light',
3 };
4
5 settings.theme = 'dark'; // OK
6 settings = {};           // TypeError
```

settings

binding



{ theme: "dark" }

объект можно менять

НУЖНА НЕИЗМЕНЯЕМОСТЬ?

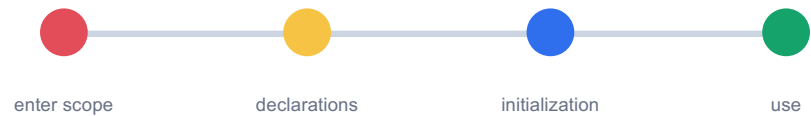
Создавайте новое значение через spread или используйте `Object.freeze()` осознанно. Freeze — поверхностный.

Hoisting: объявления известны раньше, чем инициализированы

hoisting.js

```
1 console.log(a); // undefined
2 var a = 1;
3
4 console.log(b); // ReferenceError
5 let b = 2;
6
7 sayHi(); // работает
8 function sayHi() { console.log('hi'); }
```

СОЗДАНИЕ SCOPE



var

Создаётся и сразу получает undefined.

let / const

Создаются, но до строки объявления находятся в TDZ.

FUNCTION DECLARATION

Имя и тело функции доступны до текстового объявления. Function expression ведёт себя как переменная.

Семь примитивов и один большой мир объектов

string

текст

number

числа IEEE-754

bigint

целые произвольной длины

boolean

true / false

undefined

значение отсутствует

null

явное «ничего»

symbol

уникальный идентификатор

object

Объекты, массивы, функции, Date, Map, Set...

Составные значения хранят свойства и передаются через ссылки.

 `typeof.js`

```
1 typeof 42           // "number"
2 typeof 42n          // "bigint"
3 typeof null         // "object" ← историческая ошибка
4 typeof []           // "object"
5 typeof function(){} // "function"
```

Тип принадлежит значению, а не переменной.

Динамическая типизация не отменяет явные контракты

```
conversion.js
1 let value = 42;
2 value = '42';           // допустимо
3
4 const count = Number(input.value);
5 const label = String(count);
6 const enabled = Boolean(flag);
7
8 if (Number.isNaN(count)) {
9   throw new TypeError('count must be numeric');
10 }
```

Number()

Явное
преобразование
в число.

String()

Явное
строковое
представление.

Boolean()

Проверка truthy /
falsy.

НЕ ПОЛАГАЙТЕСЬ НА УГАДЫВАНИЕ

`+` может складывать или конкатенировать. Данные из DOM, URL и JSON приходят не всегда в ожидаемом типе.

КОНТРАКТ НА ГРАНИЦЕ

Преобразуйте и валидируйте входные данные один раз — дальше работайте с предсказуемым типом.

number — floating point; NaN тоже имеет тип number

numbers.js

```
1 0.1 + 0.2 === 0.3      // false
2 Number.isNaN(0 / 0)    // true
3 Number.isFinite(1 / 0)  // false
4 Number.MAX_SAFE_INTEGER // 9007199254740991
5
6 const huge = 10n ** 30n; // BigInt
7 // huge + 1              // TypeError
```

0.1 + 0.2

Десятичные дроби не всегда
представимы точно в
двоичной форме.

NaN

«Не число» — специальное
числовое значение. Проверьте
Number.isNaN.

SAFE INTEGER

За пределами $\pm(2^{53}-1)$ целые
number теряют точность.

BigInt

Точные большие целые. Нельзя
напрямую смешивать с number.

ДЕНЬГИ

Храните минимальные единицы как целые или используйте
специализированную decimal-библиотеку.

По умолчанию используем ===; coercion должен быть осознанным

===

Строгое равенство

Сравнивает тип и значение.

4 === "4" → false
null === undefined → false
NaN === NaN → false

==

Абстрактное равенство

Сначала выполняет преобразование типов.

0 == "0" → true
null == undefined → true
[] == false → true

Object.is

Похоже на ===, но

Object.is(NaN, NaN) → true
Object.is(0, -0) → false

Полезен для особых числовых случаев.

Рабочее правило: === и !== почти всегда. == — только когда вы можете объяснить таблицу преобразований.

&&, || и ?? возвращают операнды, а не обязательно boolean

FALSY VALUES

false

0

-0

0n

""

null

undefined

NaN

||

Первое truthy значение.
Подменит 0, "" и false.

??

Правое значение только для null
или undefined.

logic.js

```
1 const title = input || 'Untitled';
2 const retries = config.retries ?? 3;
3
4 user && renderUser(user);
5 const city = user?.address?.city;
6
7 // 0 || 3 → 3
8 // 0 ?? 3 → 0
```

&&

Останавливается на первом falsy,
иначе возвращает последний
операнд.

?.

Безопасно прерывает цепочку и
возвращает undefined.

Template literals удобнее конкатенации, но строки остаются неизменяемыми

strings.js

```
1 const user = { name: 'Ada', score: 42 };
2
3 const message = `Привет, ${user.name}!
4 Баллы: ${user.score}`;
5
6 message.toUpperCase();
7 console.log(message); // исходная строка не изменилась
8
9 '😊'.length;           // 2 UTF-16 code units
10 [...'😊'].length;     // 1 code point
```

IMMUTABLE

Методы возвращают новую строку; исходная не меняется.

`${ expression }`

Интерполяция выражений и многострочный текст.

WRAPPERS

Примитивы временно получают методы. Не используйте `new String()`.

UNICODE

`length` считает UTF-16 units, поэтому emoji могут занимать 2.

ДЛЯ ЛОКАЛИЗАЦИИ

`Intl.NumberFormat` и `Intl.DateTimeFormat` лучше ручной сборки строк.

Синтаксис знаком; важнее выбрать читаемую форму

if / else

Для ветвления с разной логикой. Guard clauses уменьшают вложенность.

switch

Сравнивает через ===. Удобен для одного выражения и набора случаев.

?:

Выражение, возвращающее значение. Не превращайте в многоступенчатую логику.

guards

Ранний return/throw делает основной сценарий линейным.

flow.js

```
1 function getLabel(order) {  
2   if (!order) return 'Нет заказа';  
3   if (order.cancelled) return 'Отменён';  
4  
5   switch (order.status) {  
6     case 'paid': return 'Оплачен';  
7     case 'new': return 'Новый';  
8     default: return 'Неизвестно';  
9   }  
10 }
```

for...of — значения, for...in — ключи, методы — преобразования

for...of

Итерируемые значения

массивы · строки · Map · Set

Можно использовать break и continue.

for...in

Перечислимые ключи объекта

Не используйте для массивов: придут строковые индексы и унаследованные свойства.

array methods

map · filter · reduce · find

Описывают намерение и часто возвращают новое значение.

```
iteration.js

1 for (const order of orders) {
2   if (!order.paid) continue;
3   console.log(order.amount);
4 }
5
6 for (const key in settings) {
7   if (Object.hasOwn(settings, key)) {
8     console.log(key, settings[key]);
9   }
10 }
```

Объект — динамический набор свойств с прототипом

objects.js

```
1 const role = 'admin';
2 const user = {
3   id: 42,
4   name: 'Ada',
5   [role]: true,
6   greet() { return `Hi, ${this.name}`; },
7 };
8
9 user.name;           // dot notation
10 user['name'];         // computed access
11 Object.hasOwn(user, 'id');
```

Object.keys

Массив собственных
перечислимых ключей.

Object.values

Массив значений.

Object.entries

Пары [key, value] для
перебора.

delete / in

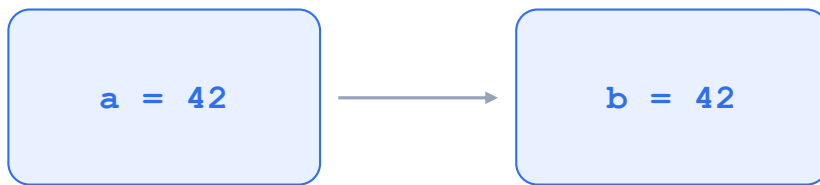
Изменяют и проверяют структуру,
но часто яснее создать новое
значение.

МЕТОД — ЭТО СВОЙСТВО-ФУНКЦИЯ

Особое поведение this определяется способом вызова, а не местом хранения функции.

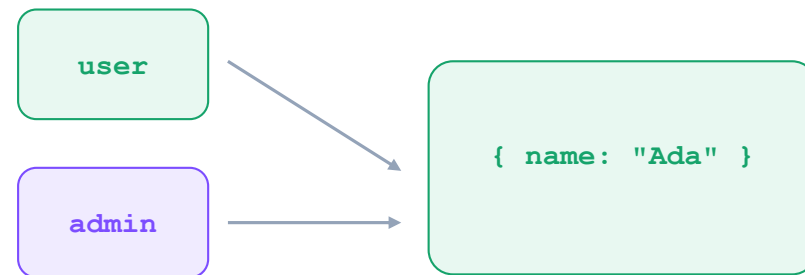
Примитивы копируются как значения; объекты — как ссылки

PRIMITIVE



две независимые копии

OBJECT



две ссылки на один объект

references.js

```
1 const admin = user;
2 admin.name = 'Grace';
3 console.log(user.name); // Grace
4
5 const copy = { ...user }; // shallow copy
6 copy.name = 'Lin';
7 console.log(user.name); // Grace
8
9 user === admin; // true
10 user === copy; // false
```

Destructuring, rest и spread уменьшают служебный код

destructuring.js

```
1 const user = { id: 42, name: 'Ada', role: 'admin' };
2
3 const { name, role = 'guest', ...meta } = user;
4 const [first, second, ...rest] = items;
5
6 const updated = {
7   ...user,
8   role: 'editor',
9 };
10
11 function render({ name, role }) {
12   return `${name}: ${role}`;
13 }
```

DESTRUCTURING

Извлекает нужные поля и позволяет переименовывать.

REST ...

Собирает оставшиеся значения в массив или объект.

SPREAD ...

Разворачивает элементы или свойства в новое значение.

DEFAULT

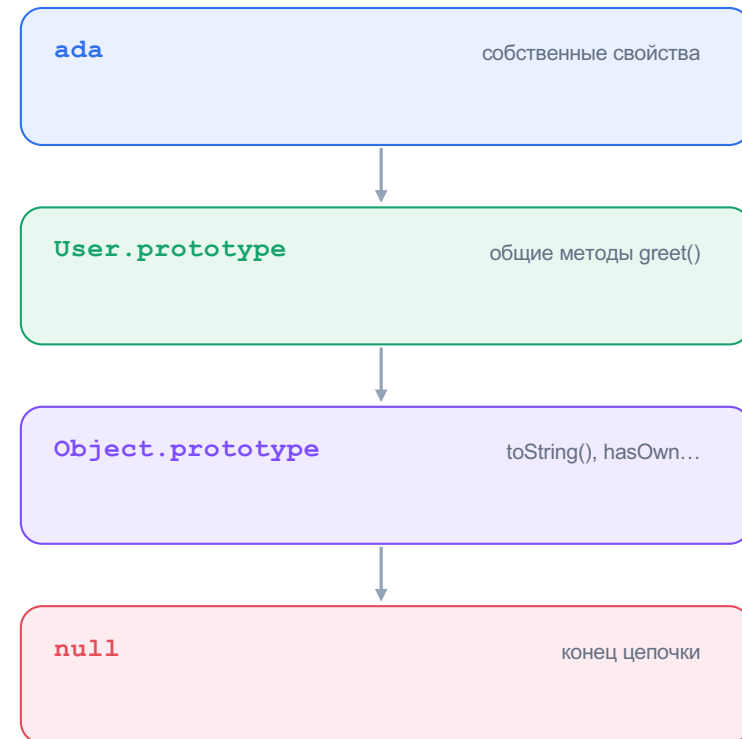
Значение по умолчанию срабатывает только для undefined.

ВАЖНО

Spread копирует только один уровень и вызывает getters. Это синтаксис, а не глубокий clone.

Наследование в JavaScript строится через prototype chain

```
class.js  
  
1 class User {  
2   constructor(name) {  
3     this.name = name;  
4   }  
5  
6   greet() {  
7     return `Hi, ${this.name}`;  
8   }  
9 }  
10  
11 const ada = new User('Ada');  
12 ada.greet();
```



class — удобный синтаксис; методы всё равно живут в prototype.

Array — объект с индексами, length и специальными методами

arrays.js

```
1 const items = ['a', 'b', 'c'];
2 items[0];           // 'a'
3 items.length;       // 3
4 Array.isArray(items); // true
5
6 items.push('d');     // mutates
7 const next = [...items, 'e'];
8
9 const sorted = items.toSorted();
10 const removed = items.toSpliced(1, 1);
```

МУТИРУЮТ

push / pop

shift / unshift

splice

sort / reverse

ВОЗВРАЩАЮТ НОВОЕ

map / filter

slice / concat

toSorted

toSpliced / with

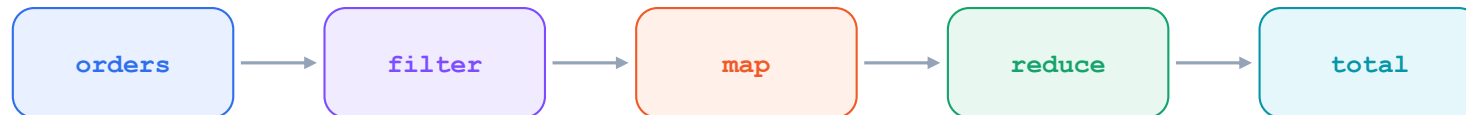
РАЗРЕЖЕННЫЕ МАССИВЫ

delete items[1] оставляет «дыру». Для удаления используйте методы массива или создавайте новый массив.

Методы массива описывают намерение как pipeline

pipeline.js

```
1 const total = orders
2   .filter(order => order.status === 'paid')
3   .map(order => order.amount)
4   .reduce((sum, amount) => sum + amount, 0);
```



find

первый элемент

some

существует хотя бы один

every

все удовлетворяют

forEach

side effects; не создаёт новый массив

Функции — значения первого класса

DECLARATION

```
function sum(a, b) {
  return a + b;
}
```

Поднимается вместе с телом. Имеет собственный this.

EXPRESSION

```
const sum = function(a, b) {
  return a + b;
};
```

Создаётся при выполнении выражения.

ARROW

```
const sum = (a, b) => a + b;
```

Короткий синтаксис. Нет собственного this и нельзя вызвать через new.

Функцию можно сохранить в переменную, передать как аргумент и вернуть из другой функции.

Параметры: default, rest, destructuring и передача значений

```

1 function formatOrder(
2   { id, amount, currency = 'RUB' },
3   ...tags
4 ) {
5   return {
6     label: `#${id}: ${amount} ${currency}`,
7     tags,
8   };
9 }
10
11 formatOrder(order, 'paid', 'priority');

```

DEFAULT

Используется, только если аргумент равен undefined.

REST

Собирает оставшиеся аргументы в настоящий массив.

RETURN

Без return функция возвращает undefined.

OBJECT PARAM

Имена полей делают вызов устойчивее к порядку аргументов.

ПЕРЕДАЧА АРГУМЕНТОВ

Всегда по значению. Для объекта этим значением является ссылка, поэтому функция может изменить тот же объект.

Функция может управлять поведением другой функции

```
callbacks.js

1 function withLogging(fn) {
2   return (...args) => {
3     console.log('call', args);
4     return fn(...args);
5   };
6 }
7
8 const double = value => value * 2;
9 const loggedDouble = withLogging(double);
10
11 button.addEventListener('click', event => {
12   console.log(event.currentTarget);
13 });
```

CALLBACK

Функция, переданная для последующего вызова.

HOF

Функция принимает или возвращает другую функцию.

SYNC

map/filter вызывают callback прямо во время обработки.

ASYNC / EVENT

Timer или событие вызовет callback позже через очередь.

НЕ ВЫЗЫВАЙТЕ СРАЗУ

Передать: `addEventListener("click", handle)`. Вызвать сейчас: `addEventListener("click", handle())`.

Функция сохраняет доступ к лексическому окружению

closure.js

```
1 function createCounter(start = 0) {  
2   let value = start;  
3  
4   return function next() {  
5     value += 1;  
6     return value;  
7   };  
8 }  
9  
10 const next = createCounter(10);  
11 next(); // 11  
12 next(); // 12
```

createCounter scope

value = 12

next()

внешняя функция завершилась,
но окружение живёт, пока доступно next

FACTORY

создание
настроенных
функций

STATE

инкапсуляция
данных

CALLBACK

доступ к
контексту позже

this у обычной функции определяется способом вызова

```

this.js

1 const user = {
2   name: 'Ada',
3   say() { console.log(this.name); },
4 };
5
6 user.say();           // Ada
7
8 const detached = user.say;
9 detached();           // undefined / error in strict mode
10
11 const bound = user.say.bind(user);
12 bound();              // Ada
  
```

obj.method()

this = obj

fn.call(x)

this = x

new Fn()

this = новый объект

arrow

нет собственного this; берёт снаружи

ПРАКТИКА

Методы объекта обычно пишите обычной функцией. Arrow удобна для callback-а, которому нужен внешний this.

Ошибки — часть контракта, а не просто console.log

errors.js

```
1 function parseAmount(value) {  
2   const amount = Number(value);  
3  
4   if (!Number.isFinite(amount)) {  
5     throw new TypeError('Invalid amount');  
6   }  
7  
8   return amount;  
9 }  
10  
11 try {  
12   save(parseAmount(input));  
13 } catch (error) {  
14   showError(error.message);  
15 } finally {  
16   stopSpinner();  
17 }
```

throw

Останавливает текущий путь выполнения и передаёт Error выше.

catch

Обработывайте только то, что можете осмысленно восстановить или сообщить.

finally

Очистка ресурсов и состояния независимо от результата.

Error

Сохраняет message, stack и может содержать cause.

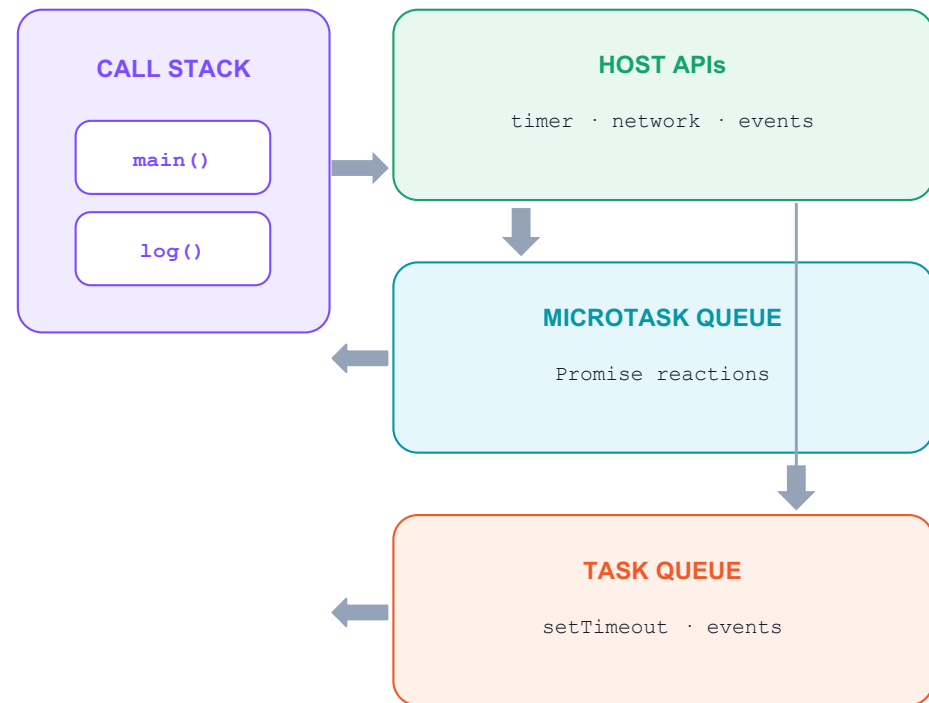
НЕ ГЛОТАЙТЕ ОШИБКУ

Пустой catch скрывает проблему. Добавьте контекст и либо восстановитесь, либо пробросьте ошибку выше.

Стек должен освободиться, прежде чем выполнится следующая задача

```
event-loop.js

1 console.log('A');
2
3 setTimeout(() => console.log('B'), 0);
4
5 Promise.resolve()
6   .then(() => console.log('C'));
7
8 console.log('D');
9
10 // A → D → C → B
```



После текущей задачи сначала очищается очередь microtasks, затем берётся следующая task.

Promise описывает будущий результат; await делает цепочку читаемой

async.js

```
1 async function loadUser(id) {  
2   try {  
3     const response = await fetch(`/api/users/${id}`);  
4  
5     if (!response.ok) {  
6       throw new Error(`HTTP ${response.status}`);  
7     }  
8  
9     return await response.json();  
10  } catch (error) {  
11    console.error('loadUser failed', error);  
12    throw error;  
13  }  
14 }
```

pendingрезультат ещё
неизвестен**fulfilled**получено
значение**rejected**получена
ошибка

fetch — API среды; Promise, async и await — механизмы языка.

async

Всегда возвращает Promise.

awaitПриостанавливает async-
функцию, но не блокирует
event loop.

Предскажите вывод — затем проверим в консоли

1 · references

```
1 const a = { value: 1 };
2 const b = a;
3 b.value = 2;
4 console.log(a.value);
```

2 · defaults

```
1 console.log(0 || 42);
2 console.log(0 ?? 42);
3 console.log(Boolean('0'));
```

3 · closure

```
1 const make = () => {
2   let x = 0;
3   return () => ++x;
4 };
5 const next = make();
6 console.log(next(), next());
```

4 · event loop

```
1 console.log('A');
2 setTimeout(() => console.log('B'));
3 Promise.resolve().then(() => console.log('C'));
4 console.log('D');
```

4 минуты · обсуждаем не только ответ, но и механизм

Обработайте данные заказов без мутации

```
orders.js

1 const orders = [
2   { id: 1, user: 'Ada', amount: 900, status: 'paid' },
3   { id: 2, user: 'Lin', amount: 650, status: 'new' },
4   { id: 3, user: 'Ada', amount: 1200, status: 'paid' },
5 ];
```

ЗАДАЧА

1. Найдите сумму оплаченных заказов.
2. Соберите уникальные имена пользователей.
3. Найдите самый дорогой заказ.
4. Верните новый summary-объект.
5. Исходный массив не изменяйте.

ОГРАНИЧЕНИЯ

const · === · pure helpers

ИНСТРУМЕНТЫ

filter · map · reduce · Set

БОНУС

Добавьте async delay и обработайте ошибку.

КОД-РЕВЬЮ

Объясните каждое преобразование данных.

Итоги занятия

JavaScript — динамический язык с объектами, функциями первого класса и событийной моделью выполнения.

1

Типы и coercion требуют явных границ

`const`, `===`, `Number/String/Boolean`, `??` и `?`.

2

Объекты живут по ссылкам

`identity`, `shallow copy`, `destructuring` и `prototype chain`

3

Функции — значения

`callbacks`, `closures`, `higher-order functions` и `this`

4

Массивы удобно читать как pipeline

`filter` → `map` → `reduce`, `find/some/every`

5

Асинхронность проходит через event loop

`tasks`, `microtasks`, `Promise` и `async/await`

Спасибо! Вопросы?